

(AWS Cafe Challenge Portfolio)

Challenge 5 - Creating a Scalable and Highly Available Environment for the Café

Student ID: 2501431
Student Name: Nijal Maharjan
Group: L5CG9
Module Leader: Sarayu Gautam
Tutor: Prabin Sapkota
Submitted On: 5/02/2026

Please submit your workshop maintaining the following guidelines through the portal
opened in MST

Challenge (Cafe) Lab: Creating a Scalable and Highly Available Environment for the Café

Task 1: Inspecting your environment

Show a screenshot of the Amazon VPC console displaying the Lab VPC details, including:

- The list of subnets (Public Subnet 1, Public Subnet 2, Private Subnet 1, Private Subnet 2) with their Availability Zones visible.
- The CafeSG security group inbound rules showing which ports are open.

Details [info](#)

VPC ID
[vpc-0ef7f2920ea92d59a](#)

DNS resolution
Enabled

Main network ACL
[acl-060a05b172d9d676f](#)

IPv6 CIDR (Network border group)
-

Encryption control ID
-

State
Available

Tenancy
default

Default VPC
No

Network Address Usage metrics
Disabled

Encryption control mode
-

Block Public Access
Off

DHCP option set
[dopt-063ec49bc9e05bd1c](#)

IPv4 CIDR
10.0.0.0/16

Route 53 Resolver DNS Firewall rule groups
-

DNS hostnames
Enabled

Main route table
[rtb-0976b5a38122368b0](#)

IPv6 pool
-

Owner ID
[550053325611](#)

[Resource map](#) [CIDRs](#) [Flow logs](#) [Tags](#) [Integrations](#)**Resource map** [info](#) Show all details

VPC
Your AWS virtual network
[Lab VPC](#)

Subnets (6)
Subnets within this VPC

us-east-1a
Public Subnet 1
Private Subnet 1

us-east-1b
Public Subnet 2
Private Subnet 2

us-east-1c
Private Subnet 3

us-east-1d
Private Subnet 4

Route tables (7)
Route network traffic to resources

Private Route Table 1
Private Route Table 2
Private Route Table 3
Private Route Table 4
Public Route Table 1
rtb-0976b5a38122368b0
Public Route Table 2

Network Connections (2)
Connections to other networks

Lab IGW
nat-0b89b3b807c79c481

Details

Security group name
[c202654a5172686148692431w550053325611-Caf](#)
[eSG-hWsgGd97xW1k](#)

Security group ID
[sg-068932fc9a3343dd4](#)

Description
[Enable SSH, HTTP access](#)

VPC ID
[vpc-0ef7f2920ea92d59a](#)

Owner
[550053325611](#)

Inbound rules count
1 Permission entry

Outbound rules count
1 Permission entry

[Inbound rules](#) [Outbound rules](#) [Sharing](#) [VPC associations](#) [Related resources - new](#) [Tags](#)**Inbound rules (1)** Manage tags Edit inbound rules

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
☐	-	sgr-03dbfb08075aef142	IPv4	HTTP	TCP	80	0.0.0.0/0	-

Task 2: Updating a network to work across multiple Availability Zones

Show two screenshots:

- The VPC console NAT Gateways list showing the newly created NAT gateway in Public Subnet 2 with status Available.
- The route table associated with Private Subnet 2, showing a route for 0.0.0.0/0 pointing to the new NAT gateway.



rtb-0538d9442f287cb94 / Private Route Table 1 Actions

Details Info

Route table ID
 rtb-0538d9442f287cb94

VPC
vpc-017a7a5d5851f9826 | [Lab VPC](#)

Main
 No

Owner ID
 633115203619

Explicit subnet associations
[subnet-0dbd27193056052fd / Private Subnet 1](#)

Edge associations
–

Routes | Subnet associations | Edge associations | Route propagation | Tags

Routes (2) Both Edit routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	nat-0e19ebf781bb1d8a5	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

Task 3: Creating a launch template

Show a screenshot of the EC2 Launch Templates console displaying the details of your newly created launch template, with the following visible:

- AMI: Cafe WebServer Image
- Instance type: t2.micro
- Security group: CafeSG
- IAM instance profile: CafeRole (visible in Advanced details)
- Resource tag: Name = webserver

Success
Successfully created [CafeTemplate\(lt-02e51d16277eb12e3\)](#).

► Actions log

Next Steps

Launch an instance

With On-Demand Instances, you pay for compute capacity by the second (for Linux, with a minimum of 60 seconds) or by the hour (for all other operating systems) with no long-term commitments or upfront payments. Launch an On-Demand Instance from your launch template.

[Launch instance from this template](#)

Create an Auto Scaling group from your template

Amazon EC2 Auto Scaling helps you maintain application availability and allows you to scale your Amazon EC2 capacity up or down automatically according to conditions you define. You can use Auto Scaling to help ensure that you are running your desired number of Amazon EC2 instances during demand spikes to maintain performance and decrease capacity during lulls to reduce costs.

[Create Auto Scaling group](#)

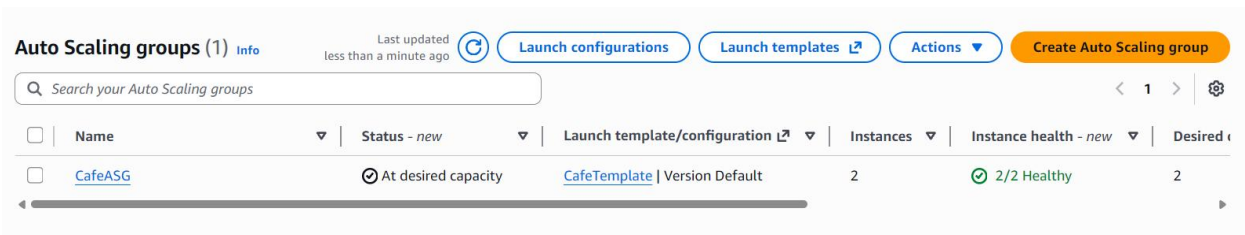
Create Spot Fleet

A Spot Instance is an unused EC2 instance that is available for less than the On-Demand price. Because Spot Instances enable you to request unused EC2 instances at steep discounts, you can lower your Amazon EC2 costs significantly. The hourly price for a Spot Instance (of each instance type in each Availability Zone) is set by Amazon EC2, and adjusted gradually based on the long-term supply of and demand for Spot Instances. Spot instances are well-suited for data-analysis, batch jobs, background processing, and optional tasks.

Task 4: Creating an Auto Scaling group

Show two screenshots:

- The Auto Scaling group details page showing: the launch template selected, VPC/subnets (Private Subnet 1 and Private Subnet 2), and group size (Desired: 2, Min: 2, Max: 6).

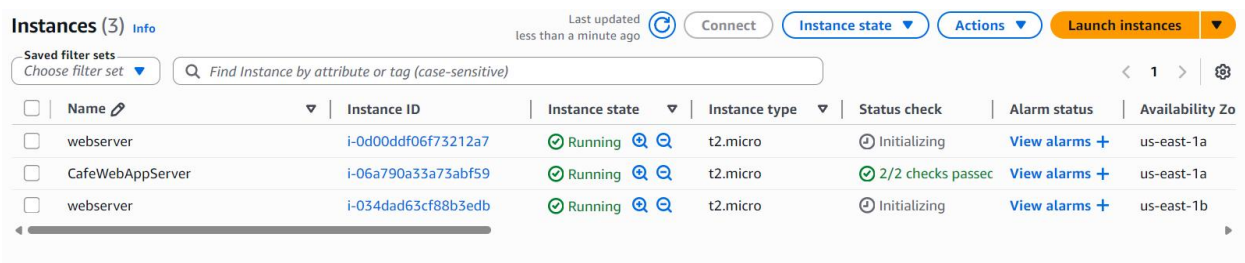


Auto Scaling groups (1) Info Last updated less than a minute ago [Launch configurations](#) [Launch templates](#) [Actions](#) [Create Auto Scaling group](#)

Search your Auto Scaling groups

☐	Name	Status - new	Launch template/configuration	Instances	Instance health - new	Desired
☐	CafeASG	At desired capacity	CafeTemplate Version Default	2	2/2 Healthy	2

- The EC2 Instances console listing the two running webserver instances launched by the Auto Scaling group, each tagged with the name **webserver**.



Instances (3) Info Last updated less than a minute ago [Connect](#) [Instance state](#) [Actions](#) [Launch instances](#)

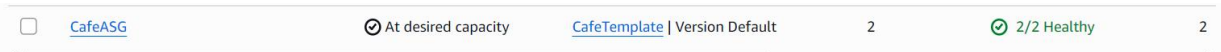
Saved filter sets: Choose filter set

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
☐	webserver	i-0d00ddf06f73212a7	Running	t2.micro	Initializing	View alarms	us-east-1a
☐	CafeWebAppServer	i-06a790a33a73abf59	Running	t2.micro	2/2 checks passed	View alarms	us-east-1a
☐	webserver	i-034dad63cf88b3edb	Running	t2.micro	Initializing	View alarms	us-east-1b

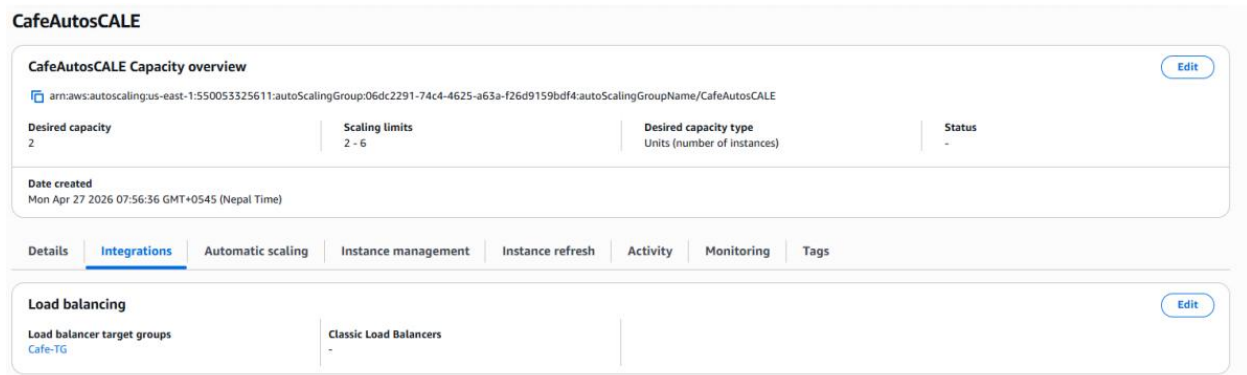
Task 5: Creating a load balancer

Show two screenshots:

- The EC2 Load Balancers console showing the Application Load Balancer in Active state, with its DNS name visible, placed in the two public subnets.



- The Auto Scaling group's Load balancing section (under Details tab) confirming the target group has been attached to the group.



 Auto Scaling group updated successfully 

Auto Scaling groups (1/1) [Info](#)
Last updated less than a minute ago 

[Launch configurations](#) [Launch templates](#) [Actions](#) [Create Auto Scaling group](#)

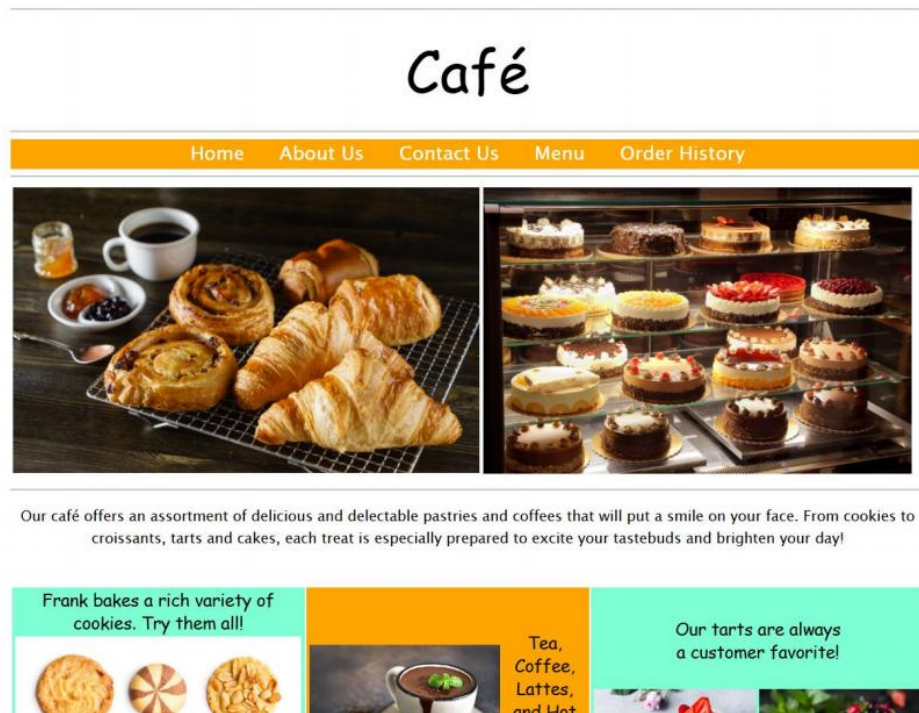
☒

Name	Status - <i>new</i>	Launch template/configuration ↗	Instances	Instance health - <i>new</i>	De
☒ CafeASG	 At desired capacity	CafeTemplate Version Default	2	 2/2 Healthy	2

Task 6: Testing load balancing and automatic scaling

Task 6.1: Testing the web application without load

- Show a screenshot of the browser successfully loading the café application via the load balancer DNS name (<http://<load-balancer-dns>/cafe>), with the full café menu visible.



Task 6.2: Testing automatic scaling under load

Show two screenshots taken after running the stress test (`stress --cpu 1 --timeout 600`) on one of the web server instances:

- The EC2 console Instances list showing more than two webserver instances in the running state (proof that the Auto Scaling group scaled out in response to the load).
- The Auto Scaling group Activity tab showing the scale-out activity history entry, confirming new instances were launched due to the CPU utilization target tracking policy being triggered.

Instances (7) info

Connect Instance state Actions Launch instances

Saved filter sets Choose filter set Find instance by attribute or tag (case-sensitive)

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
☐	webserver	i-04d077278d8e5cb9c	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1a	-	-	-	-
☐	CafeWebAppS...	i-0b1fe8c6c90cbf7c	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1a	ec2-54-197-9-219.com...	54.197.9.219	-	-
☐	webserver	i-07f51313473e26d37	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1a	-	-	-	-
☐	webserver	i-088c265c6b539e1a7	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1a	-	-	-	-
☐	webserver	i-07df1ac2ede25c0f6	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1b	-	-	-	-
☐	webserver	i-0a015166d0a8378cc	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1b	-	-	-	-
☐	webserver	i-03be8816801004aa0	Running	t2.micro	initializing	View alarms +	us-east-1b	-	-	-	-

No notifications are currently specified

Create notification

Activity history (3)

Filter activity history Filter by a date and time range

Status	Description	Cause	Start time	End time
Successful	Updating load balancers/target groups: Successful. Status Reason: Added: arn:aws:elasticloadbalancing:us-east-1:550053325611:target-group/Cafe-TG/9eabb3de6fb2fa58 (Target Group).		2026 April 27, 08:13:24 AM +05:45	2026 April 27, 08:13:25 AM +05:45
Successful	Launching a new EC2 instance: i-07df1ac2ede25c0f6	At 2026-04-27T02:11:36Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2026-04-27T02:11:37Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.	2026 April 27, 07:56:39 AM +05:45	2026 April 27, 07:57:11 AM +05:45
Successful	Launching a new EC2 instance: i-04d077278d8e5cb9c	At 2026-04-27T02:11:36Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2026-04-27T02:11:37Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.	2026 April 27, 07:56:39 AM +05:45	2026 April 27, 07:57:11 AM +05:45

Description

Write in your own wording and clearly cover all of the following (about 250 to 450 words):

1. what you built and what you did in that lab
2. one security control you used and why it matters
3. one performance or reliability feature that is present and how it helps

4. one issue you had and how you fixed it (or one likely issue and how you would fix it)

Description

A highly accessible and scalable web environment was set up for a cafe in anticipation of increased website traffic for this lab. To make sure the VPC and Security Groups were set up correctly, the initial network configuration was audited. Next, a NAT Gateway was provisioned to the public subnet and the route tables for the

In order to enable safe communication between internal servers and the internet for updates, private subnets were modified.

A Launch template was made using "Cafe WebServer Image," and the Auto Scaling group was started from it. Lastly, it was linked to an Application Load Balancer. This ensures that if one server/data center goes

down the website is still available, and when the website receives more traffic than normal the system automatically adds more servers to balance the increased load.

The deployment of the web servers on private subnets was one security measure used in this experiment. This is crucial since the servers cannot be directly accessed from the internet; instead, the Load Balancer filters all requests, effectively protecting the servers. Web servers on private subnets could still access the internet for security updates and other software thanks to the NAT Gateway.

The Auto Scaling and Application Load Balancer configuration's main dependability feature. The load balancer's continuous monitoring guarantees that only operational servers receive the traffic.

During the stress test the CPU utilization on the servers was observed and as one server was intentionally brought to a high usage, new instances were launched and sent online to share the load. This design protects the website from crashing and provides every customer to experience a smooth and well-performing site.

The inability to load the website using the Load Balancer DNS name is one potential issue. I ensured that the load balancer was situated in the public subnets rather than the private ones in order to resolve this. An LB cannot be accessed via the internet if it is located inside a private subnet. In order to access it via the LB, it was also ensured that the "CafeSG" security group has HTTP on Port 80. To maintain the site's dependability, the Auto Scaling group was configured to have at least two servers in different Availability zones.